

optimised for throughput, meaning it tries to maximise the amount of work done in a given time, even if that means longer pause times. This collector is often used for applications running on multicore processors that can afford occasional interruptions.

Concurrent Mark Sweep (CMS) GC: CMS was designed to minimise pause times by performing most of its work concurrently with the application threads. It marks live objects while the application is running and sweeps away the dead ones. However, CMS can suffer from fragmentation and isn't recommended for future projects since it's been deprecated and removed from newer Java versions.

Garbage First (G1) GC: G1 GC is a modern collector that strives to balance throughput and low latency. It divides the heap into regions and prioritises areas with the most garbage to collect first. G1 is adaptive, tuning its behaviour based on pause-time goals you set. It's the default collector for most contemporary Java applications and continues to evolve with each Java release.

Epsilon GC: Epsilon is a 'no-op' garbage collector. It doesn't reclaim memory—it simply allocates until the process runs out of heap space. This collector is useful for performance testing or scenarios where memory management is handled externally.

Z Garbage Collector (ZGC): ZGC is a scalable, low-latency collector designed to handle massive heaps (think multi-terabyte scale) with minimal pauses. It carries out most of its work concurrently, pausing application threads only briefly. ZGC is suitable for cloud workloads, large in-memory databases, and applications that cannot tolerate significant interruptions.

Shenandoah GC: Like ZGC, Shenandoah is focused on ultra-low pause times, even as heap sizes scale into terabytes. Shenandoah performs concurrent compaction, further reducing pause times compared to traditional collectors. It's particularly attractive for applications demanding high responsiveness.

What's new in Java 25 garbage collection?

Java 25 introduces several notable enhancements to garbage collection:

- **ZGC improvements:** ZGC now features even shorter pause times and better support for thread-local allocation buffers (TLABs), leading to more predictable latencies for real-time applications.
- **Shenandoah enhancements:** Shenandoah now supports improved handling of large objects and more efficient concurrent compaction, which reduces heap fragmentation and boosts allocation performance.
- **G1 adaptive tuning:** G1 GC introduces new adaptive heuristics to better balance throughput and pause time,

especially in workloads that fluctuate between bursty and steady-state memory allocation patterns.

- **Unified logging:** Java 25 brings unified and more granular GC logging, making it easier for developers to analyse memory behaviour and tune GC settings for their specific needs.
- **Better native memory tracking:** Enhanced monitoring tools now allow for more accurate tracking of native memory usage alongside heap consumption, which helps in diagnosing leaks and optimising performance.

Recommended command-line usage

Here are the recommended scenarios for each garbage collector in Java 25 and its command-line usage.

Serial GC

Recommended scenario: This garbage collector is best for small apps with limited CPU resources.

Command-line usage:

```
java -XX:+UseSerialGC -jar test-application-name.jar
```

Parallel Garbage Collector

Recommended scenario: Parallel GC is suitable for any multi-threaded application where throughput is a primary consideration.

Command-line usage:

```
java -XX:+UseParallelGC -jar test-application-name.jar
```

Concurrent Mark Sweep (CMS) Garbage Collector

Recommended scenario: CMS GC works mainly on the old (tenured) generation and uses multiple threads to mark and sweep unused objects.

Command-line usage:

```
java -XX:+UseConcMarkSweepGC -jar test-application-name.jar
```

G1 (Garbage First) Garbage Collector

Recommended scenario: This option is most suitable for applications that require consistent pause times and manage large memory heaps.

Command-line usage:

```
java -XX:+UseG1GC -jar test-application-name.jar
```

Z Garbage Collector (ZGC)

Recommended scenario: This GC is designed for applications